

An Overview of Deep Learning Architecture of Deep Neural Networks and Autoencoders

Mohit Sewak^{*} Sanjay K. Sahay[†] and Hemant Rathore[‡]

Abstract

Recently deep learning has shown great progress in multiple fields, but to perform optimally, it requires the adjustment of various architectural features and hyper-parameters. Moreover, deep learning could be used with multiple varieties of architecture aimed at different objectives, e.g., autoencoders are popular for un-supervised learning applications for reducing the dimensionality of the dataset. Similarly, deep neural networks are popular for supervised learning applications viz., classification, regression, etc. Besides the type of deep learning architecture, some other decision criteria and parameter selection decisions are required for determining the number of layers, size of each layer, activation and loss functions for different layers, optimizer algorithm, regularization, etc. Thus, this paper aims to cover different choices available under each of these major and minor decision criteria for building a neural network and training it optimally to effectively serve the objectives, e.g., malware detection, natural language processing, image recognition, etc.

Deep Learning, Deep Neural Networks, Autoencoders, Activation Function, Loss Function, Optimizer.

1 Introduction

According to Geoffrey Hinton (considered as the father of Deep Learning), deep learning (DL) *allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction* [1]. Due to this fact, DL has shown wide adoption in various fields with complex features, where an abstracted representation may hugely alleviate downstream processing and analytics, e.g., we applied the different layer of DNN and auto-encoder (AE) to improve the malware detection accuracy [2], [3].

DL method is a multiple level representation-learning methods, obtained by composing non-linear but simple modules that change the representation at one level (an raw input at the start level) into a representation at a higher, slightly more abstract level. The main aspect of DL is that the features in these layers are not designed by the engineers, but are learned from the data by using a general-purpose learning procedure. This formulation of DL, also called deep neural networks (DNN) is useful

^{*}BITS, Pilani, Dept. of CS & IS, Goa Campus, Goa, India, Email: p20150023@goa.bits-pilani.ac.in

[†]BITS, Pilani, Dept. of CS & IS, Goa Campus, Goa, India, Email: ssahay@goa.bits-pilani.ac.in

[‡]BITS, Pilani, Dept. of CS & IS, Goa Campus, Goa, India, Email: hemantr@goa.bits-pilani.ac.in

for the data where features are not related from a domain perspective. For text, and tempo-spatial or other series data, DL architectures like Recurrent Neural Networks [4], or its variants like Long-Short Term Memory [5], and Gated Recurrent Unit [6] are more popular. This is because, in regular DNN, backpropagation breaks down, due to the recurrent loop, which is solved by a technique called Backpropagation Through Time [7]. Similarly, for image and video, DL architectures like Convolutional Neural Networks (CNN) work better as they can understand the spatial correlation of pixel intensities in an image better, and in a more efficient manner [8].

The remainder of this paper is organized as follows. In the next section, we discuss the architecture of the DNN which determines the number of layers and neurons/nodes in each layer. Section 3 covers the different types of activation functions which convert the input of the neuron to an output, and also determine the function/activation for neurons in each layer of the considered architecture. Section 4 discusses the different types of loss function which ensures that the output is inline as per the objective by measuring and optimizing the loss between the prediction and output. Section 5 discusses how to minimize the loss efficiently and in an optimal manner while avoiding some of the pitfalls like getting stuck in local minima and ravines. Besides the four major criteria (Section 2 -5) which determines the network and its configuration as a whole, it is equally important to determine how to feed data into the network for training and hyper-parameters viz., batch size, epochs, and dropout regularization to avoid over-fitting and ensuring that the model is well trained, and has been discussed in section 6. Section 7 covers the architecture and different loss functions of the auto-encoders. Finally, in section 8 we summarize our paper.

2 Deep Neural Network Architecture: Layers and Nodes

A DNN (fig. 1) consists of Artificial Neural Network (ANN) nodes in different hierarchical layers. These layers consist of *input layer*, *hidden layers*, and the *output layer*. The number of *input* and *output layer* is fixed similar to the sequential models, and the number of nodes in these 2-layers are also fixed. The input layers contain as many nodes as the number of features in the input, and when used as a classifier, the output layer contains as many nodes as the number of classes.

There could be one or more hidden layers in the network, and the number of nodes in each hidden layer may differ. Though recently there has been some work to provide a scientific way using Genetic Algorithms [9, 10] to determine the number of hidden layers and nodes in each hidden layer, but mostly it is a matter of experience and trial-and-error. Heuristics do exist, which suggests that more the non-linearity and complexity in the data, more should be the number of layers to extract the important representations from such non-linear/complex data. A 2-hidden layer network is capable to represent an arbitrary decision boundary to certain accuracy with rational activation functions and could approximate any smooth mapping with the accuracy [11].

The heuristics approach on the number of nodes in a hidden layer suggests that, these number of nodes should remain between the number of input and output (or the next layer) layer nodes. Some heuristics gives a roughly around two-third number of nodes of the input layer, plus the number of nodes of the output layer, others suggest

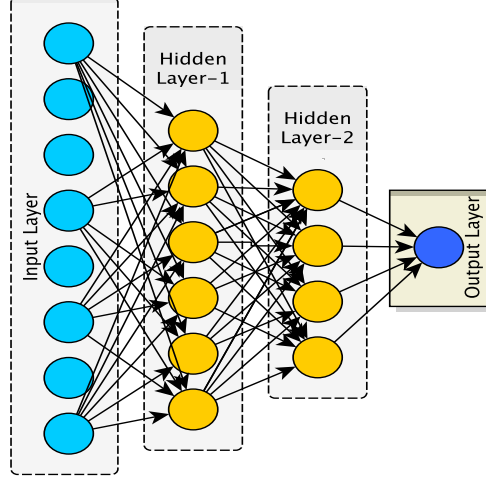


Figure 1: A schematic of a sequential DNN with two hidden layers.

an upper bound of two times the number input layer nodes for the number of hidden layer nodes. [11].

3 Activation Functions

In ANN activation function is used to convert an input signal of a node to an output signal, and can be given as

$$Y = \text{Activation}(W^T X + c)$$

where,

Y = Output,

W^T = Transpose of the weight vector W of the neuron,

X = Input vector to the neuron, and

c = bias of the neuron.

For example, if we have a dataset containing n records $(R_1, \dots, R_n)$, and each record contains m features $(X_{r,1}, \dots, X_{r,m})$, where r is the r^{th} record in the dataset. Then taking an instance of the i^{th} record, whose features are given by a $[1, m]$ matrix $[X_{i,1}, \dots, X_{i,m}]$, which is an input to the n^{th} (say) neuron in the l^{th} layer represented by $N_{l,n}$, whose weight vector W is given by another $[1, m]$ matrix $(W_1, \dots, W_m)$, with a bias of $b_{l,n}$, then the output $Y_{i,(l,n)}$ for the i^{th} record from this neuron under a given activation function f_{n_A} is given by

$$Y_{i,(l,n)} = f_{n_A}(W_1 X_{i,1} + \dots + W_m X_{i,m} + b_{l,n})$$

Since training in neural networks happens through backpropagation, which requires the activation function should be differentiable in the entire range of input. Another

requirement for the activation function is that it should be monotonic (continuously increasing or decreasing in the range of the input).

The activation function for hidden and inputs layers are mostly different from that of the output layer, as the primary function of the activation function in the input or hidden layers is to enable non-linearity, whereas that of the output layer of a DNN used for classification is to give standardized class probabilities for the each class. Some of the popular activation functions used in DL are Softmax [12], Sigmoid [13], Tanh [13], Linear (Identity) [14], Rectifier Linear Unit [15] and Exponential Linear Unit [16].

3.1 Softmax Activation

As discussed above, the output layer for a classification, DNN should provide the class probability for each class under consideration, and for this a softmax function is used for its activation. As per softmax function, the probability of the output y to assume a particular class c out of total K classes, provided the input feature vector x is given as

$$P(y = c|x) = \frac{e^{x^T w_j}}{\sum_{k=1}^K e^{x^T w_k}}$$

where, x^T is the transpose of the weight vector x .

3.2 Sigmoid and Tanh Activations

There is a lot of variation available for the activation of input and hidden layers. In traditional artificial neural networks sigmoid, hyperbolic tangent activation were used, which is given as

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

$$\text{Tanh}(x) = \frac{2}{1 + e^{-2x}} - 1$$

If these activation were used in the *hidden layers*, it will be difficult to train a DNN, because the neurons would get stuck in the upper and lower areas of sigmoid and hyperbolic tangent functions. This is due to the vanishing or exploding gradient problem in such networks. This drawbacks contributes to the fact that DL could not be practically adopted to solve complex problems until new activation functions were discovered. In this, sigmoid functions could also be used as activation function for the last layer, e.g., in the binary classification.

3.3 Linear and Identity Activation

As we discussed earlier, for classification problems sigmoid (binary classification), and softmax (multinomial classification) activation could be used, as these provide steep non-linear separation between the classes. But for regression problems (predicting continuous variable output), Linear activation may be required. Identity activation falls

in the class of linear activation and is used in DL as activation for the last layer for a regression problem for predicting a continuous output [14].

$$\text{Identity}(x) = \sum_i x_i w_i$$

where, x_i and w_i corresponds to the i^{th} element of input and weight vectors.

3.4 Rectifier Linear Unit Activation

Rectifier Linear Unit (ReLU) activation, solved the vanishing and exploding gradient problem as highlighted above as the derivative of a unit for positive values [17], and many of the popular networks use this as the activation for input/ hidden layers. ReLU function behaves like a identity function $f(x) = x$ for all positive x , and 0 otherwise, given as

$$\text{ReLU}(x) = \max(0, x)$$

Though the differential of ReLU is not a continuous function, it is defined in the entire range of input (1 for positive x, and 0 for otherwise). As ReLU continues to be the most popular activation function for most of the DL architecture, hence variants of ReLU are developed, such as

3.4.1 Leaky ReLU

In ReLU, the differential for negative x is defined as 0, that means no gradient for this range of x, which makes training through backpropagation challenging in the absence of a gradient. Leaky ReLU [18] tries to solve this by changing ReLU slightly as

$$\text{LeakyReLU}(x) = \begin{cases} ax & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

where, a is a constant, usually taken 0.01.

3.4.2 Randomized Leaky ReLU

In the above formulation for Leaky ReLU, the value of a is fixed. If instead it is allowed to change randomly, it is called Randomized Leaky Rectifier Linear Unit (RRReLU)[19], and in such a case, its range becomes $(-\infty, \infty)$. Both of the above derivatives of ReLU are not only continuous and monotonic but also have a monotonic gradient.

3.4.3 Parametrized ReLU

In Parametrized Rectifier Linear Unit (PReLU) [20], as the name suggests, the parameter a of the Leaky ReLU is optimized during the training processes.

3.5 Exponential Linear Unit

Although the above variants of ReLU, does solve the problem of dead or not activated neurons, there still exists another problem in them, which is mean shift. Since the activations are predisposed to positive outputs, their mean is not zero centered, which cause a problem in learning. To solve this problem, either a Batch Normalization [21] layer could be applied after the activation, or else, Exponential Linear Unit (ELU) could provide a computationally more efficient solution to overcome this drawback, and is defined for positive α as

$$ELU(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

The value to which an ELU saturates for negative net inputs is controlled by the ELU hyperparameter α . The sigmoid and tanh function are contractive almost everywhere, but in ELU, since it's gradient is unity it is not contractive. ELU is also able to overcome the vanishing gradient problem as the positive part of the gradient this function is identity.

After configuring the architecture of DNN, the activation/ function of the neurons in each layer of the architecture has been determined. Therefore, this section covers the different types of activation functions which convert the input of the neuron to output and for the intermediate layer. To ensure that the output is inline as per the objective, in the next section we discuss the loss function that can be used to measure and optimize the loss between the prediction and output under different conditions.

4 Loss Functions

The objective to train a neural network is to identify the weights (and other parameter values) that minimizes the loss between the expected and the predicted output of a neural network, as determined by its loss function. Most of the loss functions as used in different popular machine learning algorithms could also be used in DL, the applicability of the same depends upon the target function distributions. In case of categorical target variable, e.g., in binary classification, commonly used loss functions are [22]

- L1 loss: $\|y - o\|$
- L2 loss: $\|y - o\|^2$
- Expectation Loss: $\|y - p(o)\|$
- Regularized Expectation Loss: $\|y - \sigma(o)\|^2$
- Hinge Loss: $\sum_j \max(0, \frac{1}{2} - \hat{y}_j o_j)$
- Squared Hinge Loss: $\sum_j \max(0, \frac{1}{2} - \hat{y}_j o_j)^2$
- Cross Entropy Loss: $-\sum_j y_j \log \sigma(o)_j$

- Squared log Loss: $-\sum_j [y_j \log \sigma(o)_j]^2$

where,

y = the true output/label vector (one hot encoded) consisting of j elements,

$\hat{y}$ = true output label (categorical/class),

o = the output vector of the last layer (consisting of j elements), and

$\sigma(o)$ = expectation of the probability distribution of output vector o .

Of the above loss functions, the cross-entropy loss (binary cross entropy for binary classification and categorical cross-entropy for multi-class classification) is the most popular, though some experiments indicates that hinge loss [23] and squared hinge loss [24] could also be a good candidate to use as a loss function in DL. Some studies also indicate that expectation loss might be a much better loss function to deal with the noisy data [25].

5 Optimizers

In the previous section, we discussed that we need to minimize the loss for achieving a good training for the DNN. This task of minimization of the objective function is what the optimizer has to do in an ‘*optimal*’ manner. The optimizer does this by finding a good minimum for the loss function or the objective function. The computation efficiency to reach this optimization is just one of the aspect that determines the effectiveness of the optimizer. Other aspects could be how robust it is, and how well it performs under different conditions like ridges in gradient topology and skewed gradients parameters. Since most of the optimizers used in the DL are built over gradient descent. Therefore first we will discuss the gradient descent for academic understanding, and then cover some more popular optimizers built on it.

5.1 Gradient Descent

For a DNN, the loss function $J(\theta)$ is a function of the weights that need to be optimized, parameterized as a vector $\theta \in \mathbb{R}^d$. Since here optimization refers to minimization of a convex function, so we need to find the gradient of this function and move in the opposite direction of it till we find a (hopefully global) minima. The gradient with respect to the weight vector θ is defined as $\Delta_\theta J(\theta)$.

The learning rate (η) determines the correction in the opposite direction of the gradient that we do in each iteration. If the iteration happens over the complete data-set, then it is called the **Batch Gradient Descent** ($\theta = \theta - \eta \cdot \Delta_\theta J(\theta)$), else, if the correction happens in an stochastic manner over every training sample (x_i, y_i), then it is called a **Stochastic Gradient Descent** ($\theta = \theta - \eta \cdot \Delta_\theta J(\theta; x_i, y_i)$), and if the same happens over a combination of both, i.e., a mini-batch of data n (say) samples, then it is called a **Mini-Batch Gradient Descent** ($\theta = \theta - \eta \cdot \Delta_\theta J(\theta; x_{i:i+n}, y_{i:i+n})$).

5.2 Nesterov accelerated gradient

If the gradient hyper-plane curves changes with different parameters (ravines), then there may be an steep correction across some of the parameters with high gradient, thus leading to hunting (missing the local minima and repeatedly going on opposite direction of the convex slope) across the minima or conversely converging very slow in the direction of the parameter whose gradient is too small. To overcome this effect the nesterov accelerated gradient (NAG) [26], use the momentum γ , and the gradient update of the last time step (v_{t-1}) to update the current gradient (v_t) using the look-ahead gradient ($\theta - \gamma v_{t-1}$) as $v_t = \gamma v_{t-1} + \eta \Delta_\theta J(\theta - \gamma v_{t-1})$

5.3 Adaptive Gradient

Using momentum, NAG does solve the problem of uneven gradient slope across the elements of gradient hyperplane, but its an indirect approach. Adaptive Gradient (ADAGRAD) [27] does the same this directly. Unlike previous methods, which use the same learning rate η for all the elements of the vector θ , ADAGRAD updates each element i ($\theta_{t,i}$) of θ at step t differently as $\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}$, where $g_{t,i}$ is the gradient of element i at step t , and $G_{t,ii} \in \mathbb{R}^{d \times d}$ is the element ($i \times i$) of the diagonal matrix G at time t . Each element of G is given by $G_{t,ii} = \sum_{s=1}^t \theta_{s,i}^2$ and ϵ , a small constant, helps in preventing divide by zero issues. ADAGRAD has proved very effective over large DL networks as the ones used to generate word embedding for NLP [28] and by Google for classification of objects in YouTube videos [29].

5.4 ADADELTA and RMSProp

Both ADADELTA and RMSProp [30] tries to improve the ADAGRAD's weakness of working on the squares of all the gradient element across each step from the beginning, which continuously decreases the effective learning rate monotonically in an aggressive manner. Both of these proposes a fixed step/time period across which this effect should last (based on running average). ADADELTA replaces the diagonal matrix G_t with $E[g^2]_t$ given as $E[g^2]_t = \lambda E[g^2]_t + (1 - \lambda)g_t^2$, where λ is the exponential decay coefficient. RMSProp sets the value of λ as 0.9, such that $E[g^2]_t = 0.9E[g^2]_t + 0.1g_t^2$.

5.5 Adaptive Moment Estimation

Adaptive Moment Estimation (ADAM) [31] takes ideas of adaptive learning rate from ADAGRAD/ ADADELTA/ RMSProp and moment estimation for decaying the past gradient changes. It use both the first moment m_t and second moment v_t of the gradient at time t to update at each step exponentially as $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$, and $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$, where β_1 and β_2 are exponential decay coefficients for the first and second order moments. The gradient is updated in each step as $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$, where $\hat{v}_t$ and $\hat{m}_t$ are the bias corrected estimates of m_t and v_t , given

as $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$ and $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$. Here, the suggested default values for β_1 and β_2 are 0.9 and 0.999 respectively, and for ϵ is 10^{-8} .

6 Training Considerations: Batches, Epochs & Dropout

As discussed in previous section, we use mini-batches to train a DNN, wherein the whole data is divided into batches of fixed sized. A complete batch optimization would lead to sharp minima and poor generalization, and is not suitable for stochastic process, and a complete stochastic/online process makes training very noisy [32]. For effective training the other hyperparameter has to be considered are batch size, epochs and dropout regularization.

6.1 Batch-size

As deep learning requires a lot of data to train, therefore, a large batch size may not be optimal to hold in memory, and can lead to over-fitting, where as too small batches may require a large time to converge and may be noisy. A batch size of 32-512 samples [32] is found to be used in general practice in deep learning.

6.2 Epochs

As mentioned earlier, a single pass over data may not be optimal, so multiple passes over data are made for the purpose of training, and each pass over the complete data will required $\lceil \frac{n}{b_size} \rceil$ batches, where n is the number of data records and b_size is the batch size. Besides prior experience and heuristics, the number of epochs are determined seeing the convergence plots of training/test error as the epoch proceed during the training process. The weights of the network could be saved intermittently (checkpoints) and the network restarted from last checkpoint/trained-state for further epochs, if required, for further optimization of weights.

6.3 Dropout Regularization

Overfitting is an issue in machine learning, where during the training, model not only learn pattern of the data, but also start fitting the noise. Hence, not able to perform well on the test data or in real-life applications. For other machine learning algorithms like decision tree, ensembling is a very good technique to avoid overfitting, e.g., random forest use it effectively. But since DL networks are hard to train, ensembling may not be an efficient method to be applied. Therefore DL use a technique called dropout for regularization [33], in which during training, randomly some nodes from the given layer would be dropped/switched-off. The proportion of the nodes being dropped-out could be controlled by a parameter which determines the number of nodes to be dropped in a particular layer. It gives a network that has smaller weights and has a similar effect of using multiple thinned networks as in case of ensembling. Dropout is

applied only during training and all the nodes are retained during testing. Sometimes this may lead to smaller test loss than training loss for some epochs.

7 Autoencoders

Autoencoders [34] was first introduced in 1986 by Geoffrey Hinton [35] are DL architectures for unsupervised learning. They are mainly used for data compression and learning useful abstraction of data. There are generative or variational autoencoders as well which can learn latent model of the data for the purpose of generating new data, or as de-noising autoencoders to minimize the noise from the data. Autoencoders can also be applied to generic or image data also in the form of CNNs [8]. In this section we cover the autoencoders that are used for the dimension reduction.

7.1 Architecture

Autoencoder (fig. 2) consists of a network of encoder and decoder. During training both encoders and decoders are used. But when applied to new data for the reduction of dimension, then only encoder are used. Like any DL architecture, AE works in layers of neurons, and are trained using backpropagation. The layers are divided into different encoder and decoder layers. The input is connected to the first encoder layer. In each subsequent layer of the encoder, the number of neurons are reduced till we reach the last encoded layer, which has the least number of neurons, representing the bottleneck features. The input are transformed until this layer represents the reduced dimension of the data that is capable of representing the maximal signals in the data. After this layer, the decoder layers are set, in which the number of neurons are increased in each layer, until the output layer which has the same number of neurons as the input.

7.2 Loss Function

The objective while training the autoencoder is to minimize the reconstitution error between the output (the last decoder layer) and the input (the first encoder layer), which ensures that even after compressing the data through the encoders, the bottleneck layers have learn most of the relevant patterns in the data, such that it could be re-constituted again effectively after decoding. For continuous input, squared error loss could be one of the loss functions for AE, and is given as

$$\mathcal{L}(x, x') = \|x - x'\|^2 = \|x - W'(Wx + b) + b'\|^2$$

where,

x is the input,

W, b is the weight vector and bias of the encoder, and

W', b are the weight vector and bias of the decoder.

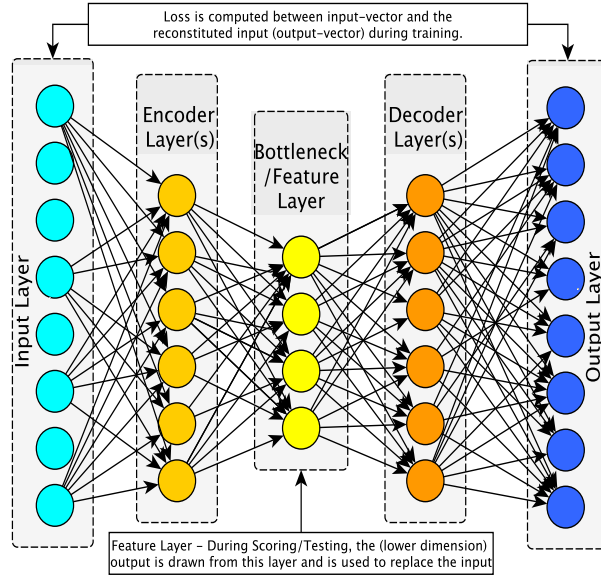


Figure 2: A schematic of an AE showing an encoder connected to the input layer leading to bottleneck layer/ features followed by decoder leading to the reconstituted output layer.

When considered in batches of n samples, the mean square error for the batch could be computed by the formulae

$$MSE = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(x_i, x'_i)$$

For categorical input, cross-entropy loss as discussed in section 4 could be used.

8 Summary

There exists work that covers individual aspects of DL like loss functions, activation, etc., but to the best of our knowledge, this is the first paper that covers all the architectural aspects of the two most basic DL networks, i.e., DNN for supervised learning and autoencoders for un-supervised learning. This paper started with step of choosing the DL type based on requirement, then logically leading to the decision of selecting the number of layers in the selected network, then structurally leading to the aspects related decisions for each layer, namely the number of nodes and activation function for each layer, then we cover the mathematical aspects for training the selected network, namely the selection of loss function and the optimizer, and then the practical aspects of the training, namely deciding the batch size for data samples and epochs to train the network. However, we also believe that it would be relevant to cover other DL

architectures like RNN, variational AE, CNN etc., and in this direction the work is in progress.

References

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436, 2015.
- [2] Mohit Sewak, Sanjay K Sahay, and Hemant Rathore. Comparison of deep learning and the classical machine learning algorithm for the malware detection. In *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, pages 293–296. IEEE, 2018.
- [3] Mohit Sewak, Sanjay K Sahay, and Hemant Rathore. An investigation of a deep learning based malware detection system. In *Proceedings of the 13th International Conference on Availability, Reliability and Security*, page 26. ACM, 2018.
- [4] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning, ICML '06*, pages 369–376, New York, NY, USA, 2006. ACM.
- [5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [6] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [7] Manuel P Cuéllar, Miguel Delgado, and MC Pegalajar. An application of non-linear programming to train recurrent neural networks in time series prediction problems. In *Enterprise Information Systems VII*, pages 95–102. Springer, 2007.
- [8] Mohit Sewak, Md Rezaul Karim, and Pradeep Pujari. *Practical Convolutional Neural Networks: Implement advanced deep learning models using Python*. Packt Publishing Ltd, 2018.
- [9] Mohammed Amine Janati Idrissi, Hassan Ramchoun, Youssef Ghanou, and Mohamed Ettaouil. Genetic algorithm for neural network architecture optimization. In *Logistics Operations Management (GOL), 2016 3rd International Conference on*, pages 1–4. IEEE, 2016.
- [10] D Stathakis. How many hidden layers and nodes? *International Journal of Remote Sensing*, 30(8):2133–2147, 2009.

- [11] Jeff Heaton. *Introduction to neural networks with Java*. Heaton Research, Inc., 2008.
- [12] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006.
- [13] Kevin L. Priddy and Paul E. Keller. *Artificial Neural Networks: An Introduction (SPIE Tutorial Texts in Optical Engineering, Vol. TT68)*. SPIE- International Society for Optical Engineering, 2005.
- [14] D. F. Specht. A general regression neural network. *Trans. Neur. Netw.*, 2(6):568–576, November 1991.
- [15] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.
- [16] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). *arXiv preprint arXiv:1511.07289*, 2015.
- [17] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323, 2011.
- [18] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proc. icml*, volume 30, page 3, 2013.
- [19] Bing Xu, Naiyan Wang, Tianqi Chen, and Mu Li. Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*, 2015.
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- [21] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- [22] Katarzyna Janocha and Wojciech Marian Czarnecki. On loss functions for deep neural networks in classification. *arXiv preprint arXiv:1702.05659*, 2017.
- [23] Yichuan Tang. Deep learning using linear support vector machines. *arXiv preprint arXiv:1306.0239*, 2013.
- [24] Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. Deeply-supervised nets. In *Artificial Intelligence and Statistics*, pages 562–570, 2015.

- [25] Katarzyna Janocha and Wojciech Marian Czarnecki. On loss functions for deep neural networks in classification. *arXiv preprint arXiv:1702.05659*, 2017.
- [26] Yurii Nesterov. A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$. In *Doklady AN USSR*, volume 269, pages 543–547, 1983.
- [27] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.
- [28] Jeffrey Pennington, Richard Socher, and Christopher Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [29] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, et al. Large scale distributed deep networks. In *Advances in neural information processing systems*, pages 1223–1231, 2012.
- [30] Matthew D Zeiler. Adadelta: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.
- [31] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [32] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.
- [33] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.
- [34] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [35] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning internal representations by error propagation. Technical report, California Univ San Diego La Jolla Inst for Cognitive Science, 1985.